# Relazione di Progetto: Sistema Multi-Agente per il Recupero di Oggetti in una Rete di Magazzini

Antonio Rosano, Angelo Spadola, Marco Spampinato

Link alla repository: GitHub - AI Swarm Project.

## 1 Introduzione e Definizione del Problema

Il presente elaborato si presenta come relazione di progetto di un modello di simulazione basato su un Sistema Multi-Agente. Lo scopo del sistema realizzato è coordinare una flotta di agenti robotici autonomi incaricati di ricercare, recuperare e posizionare un insieme di oggetti dispersi in una vasta area logistica composta da una rete di magazzini.

Il dominio operativo è modellato come una griglia bidimensionale statica. Rispetto alle specifiche di base, è stata rimossa la conoscenza pregressa sul numero e sulla collocazione dei magazzini, al fine di introdurre un maggiore grado di realismo e complessità. Partendo dalla cella in alto a sinistra di coordinate [0,0], la flotta ha il compito di esplorare l'area alla cieca, individuando dinamicamente sia i punti di drop-off (magazzini) che gli oggetti da recuperare, costruendo e aggiornando la propria mappa locale in tempo reale.

L'efficienza del modello algoritmico viene valutata in base alla sua capacità di ottimizzare simultaneamente tre metriche fondamentali:

1. Massimizzare il numero di oggetti rilevati e correttamente posizionati all'interno del magazzino.
2. Minimizzare il tempo totale di recupero.
3. Minimizzare l'energia media consumata dagli agenti durante le operazioni esplorative e di trasporto.

## 2 Software realizzato

Il modello di simulazione proposto ha sua implementazione realizzata usando il linguaggio di programmazione *Python 3* e le relative librerie *numpy* per la manipolazione di array n-dimensionali, *pandas* per la manipolazione dei dati dei risultati, *matplotlib* e *rich* rispettivamente per la visualizzazione della simulazione e strumenti atti al monitoraggio di batch di simulazioni.

La soluzione implementata è struttarata nei seguenti file e cartelle:

1. **maps**: è la cartella che contiene le mappe su cui eseguire le simulazioni, fornite alla consegna del progetto.
2. **results**: è la cartella dove vengono immagazzinati i risultati delle simulazioni.
3. **src**: è la cartella dove sono immagazzinati i sorgenti *Python* del progetto:
    - **main.py**: è l'entry-point della soluzione.

- **agent.py**: è il sorgente contenente la classe *Agente*, la quale descrive lo stato di un agente all'interno della simulazione.
- **geometry.py**: il file contiene il sorgente della classe di utilità *Position* per la rappresentazione delle posizioni e relativi metodi di misura della distanza.
- **sensors.py**: il sorgente contiene le classi *CommunicationSensor* e *VisibilitySensor*, componenti dell'agente responsabili rispettivamente della comunicazione tra agenti e la visione del singolo agente.
- **strategy.py**: il sorgente responsabile delle strategie, contiene la classe astratta *BaseStrategy* implementa funzionalità e utilità di base per le diverse strategie implementate, e le varie strategie implementate.
- **map.py e parse_json.py**: sono i sorgenti responsabili, rispettivamente, della rappresentazione della mappa all'interno del software tramite la classe *Map*, e il caricamento dal file system delle mappe fornite per la simulazione.
- **Logger.py e graphics.py**: sono sorgenti contenenti le utilità rispettivamente di *logging* di eventi e risultati e visualizzazione grafica della simulazione.

Sì invita alla lettura del file **readme.md** per chiarimenti e indicazioni sull'uso del software realizzato.

# 3 Modellazione dell'Ambiente e degli Agenti

L'ambiente logistico è implementato come una griglia bidimensionale discreta, in cui ogni cella assume un valore enumerato che ne definisce la tipologia. Gli elementi strutturali (muri) delimitano le zone di transito e i corridoi, definendo una topologia di navigazione complessa ma statica. I magazzini sono dotati di entrata e uscita, rappresentati da rispettivi tipi cella, imponendo così vincoli di manovra durante la fase di *drop-off* degli item.

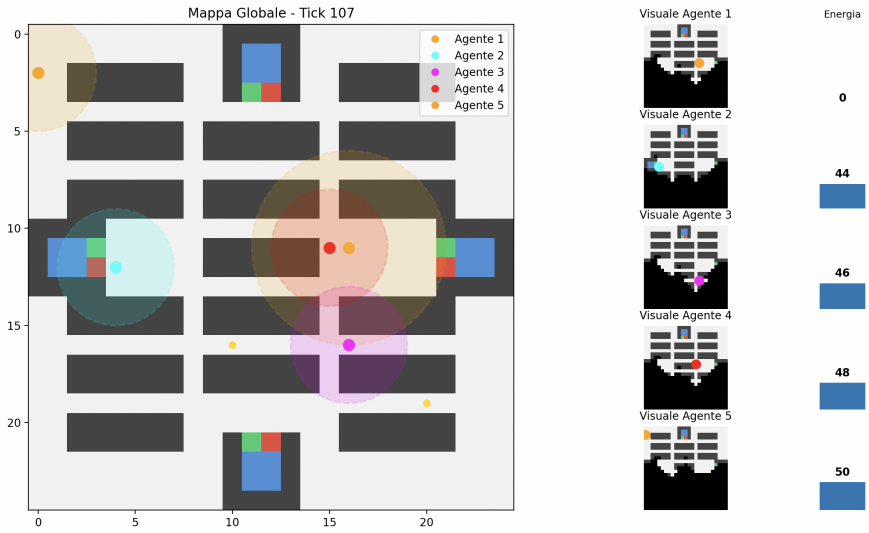


Figura 1: Rappresentazione visiva dell'ambiente di simulazione (generata tramite il modulo *graphics.py*). Sono visibili gli agenti spazialmente distribuiti, gli ostacoli strutturali, gli oggetti da recuperare e le zone di drop-off (magazzini).

All'interno di questo spazio opera una flotta di agenti autonomi, ciascuno progettato con uno stato interno e una **dotazione sensoriale specifica**. Lo stato dell'agente è definito dalle sue

coordinate spaziali, da un indicatore di carico (*carrying*) che segnala l'avvenuto recupero di un oggetto, e dal livello di energia residua, la quale decresce progressivamente ad ogni azione di movimento. Per interagire con l'ambiente circostante, ogni robot si avvale di due moduli percettivi:

- **Sensore di Visibilità:** Opera entro un raggio predefinito e permette all'agente di acquisire informazioni topologiche (ostacoli e varchi) e rilevare target (oggetti e porte del magazzino). Il sensore rileva inoltre la presenza di altri agenti nelle celle adiacenti, informazione cruciale per attivare manovre evasive ed evitare sovrapposizioni e stalli nei colli di bottiglia.
- **Sensore di Comunicazione:** Abilita alla cooperazione tra agenti. Quando la distanza euclidea tra due agenti risulta inferiore al raggio di comunicazione, essi instaurano un collegamento per sincronizzare le rispettive memorie locali. Questo scambio permette di unire le porzioni di mappa esplorate e di invalidare informazioni obsolete (come la presenza di un oggetto già recuperato da un altro componente dello sciame).

Ogni informazione captata dai sensori viene immagazzinata in una **mappa locale** individuale. Questa memoria spaziale parte da uno stato di totale incertezza e viene progressivamente popolata, fungendo da base conoscitiva fondamentale per gli algoritmi di *Decision-Making* e Pathfinding del singolo agente.

# 4 Processo Decisionale e Strategie di Esplorazione

L'architettura logica che governa il comportamento degli agenti è strutturata su due moduli interconnessi, al fine di garantire un'efficace capacità di *Decision-Making* e una scelta autonoma del percorso.

## 4.1 Architettura Comportamentale e Macchina a Stati Finiti

Il ciclo di vita operativo dell'agente è gestito da una Macchina a Stati Finiti (FSM). Lo stato predefinito è l'esplorazione (*EXPLORE*), che guida la ricerca di nuovi target. Tale stato viene interrotto non appena il sensore individua un oggetto, innescando la transizione allo stato di recupero (*REACH_ITEM*). Acquisito il carico, l'agente passa allo stato di consegna (*SEEK_STORAGE*), calcolando la rotta ottimale verso l'ingresso del magazzino. È stata inoltre implementata una logica di emergenza (*LOW_ENERGY*) che, in caso di esaurimento critico della batteria, forza l'agente verso una zona di sosta strategica.

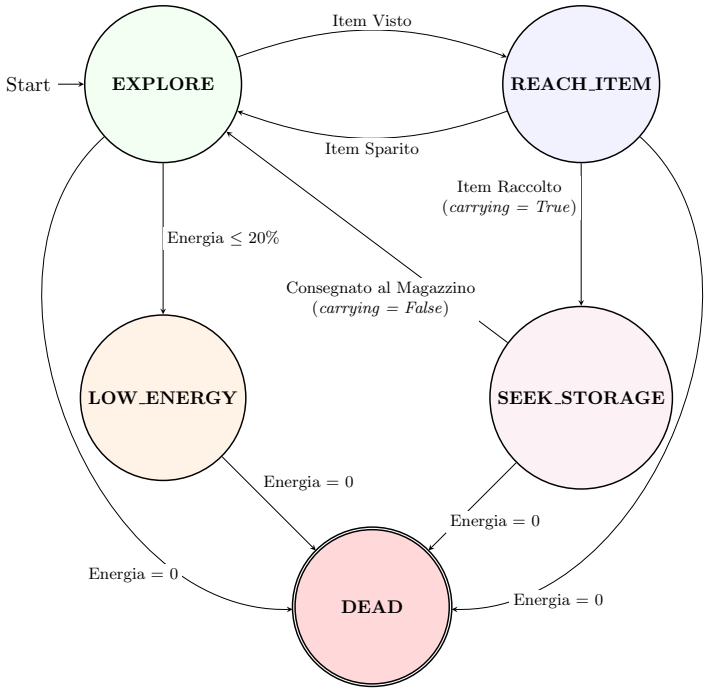


Figura 2: Macchina a Stati Finiti (FSM) che regola il comportamento del singolo agente.

## 4.2 Algoritmi di Navigazione e Risoluzione dei Conflitti

Il sistema di navigazione degli agenti si fonda su un approccio ibrido che unisce la ricerca deterministica a euristiche di memoria spaziale. Quando un agente acquisisce un target noto (un oggetto da raccogliere o un magazzino in cui depositare), il calcolo della traiettoria è affidato a un algoritmo di ricerca in ampiezza **(BFS)**. L'implementazione di quest'ultimo è stata specificamente adattata alla topologia dell'ambiente: l'algoritmo ignora i muri ed evita le celle di uscita quando l'agente cerca di entrare, e viceversa.

Per governare le fasi di ricerca cieca e prevenire percorsi ciclici ridondanti, la navigazione è supportata da una memoria a breve termine (**Tabu List**). Questa struttura dati a coda circolare (con dimensione massima prefissata) tiene in memoria gli ultimi passi effettuati dall'agente, scoraggiando il ritorno sulle celle appena visitate durante la selezione stocastica delle mosse.

Infine, per garantire la fluidità del traffico in spazi ristretti, è stato sviluppato un modulo stocastico di manovra evasiva (**Collision Event**): qualora la traiettoria di un agente converga su una cella attualmente occupata da un altro agente attivo (*deadlock* imminente), l'agente invalida immediatamente il percorso in memoria e successivamente calcola uno spostamento laterale o di arretramento, estraendo casualmente una direzione libera da ostacoli e da altri agenti, disimpegnando l'incrocio e permettendo allo sciame di riorganizzarsi fluidamente.

## 4.3 Strategie Esplorative e Intelligenza di Sciame

Il cuore decisionale del progetto risiede nel modulo strategico, implementato seguendo il **Design Pattern Strategy**. Tutte le logiche derivano da una classe astratta (*BaseStrategy*) che fornisce gli strumenti fondamentali di mappatura. Per valutare l'impatto della cooperazione, sono state sviluppate e testate logiche a complessità crescente:

- **Strategie Baseline (Random e Scout):** La **RandomStrategy** rappresenta il limite inferiore di efficienza, basandosi su un puro *Random Walk* filtrato unicamente dalla Tabu List. Per ottimizzare la copertura dei corridoi, è stata introdotta la **ScoutStrategy**, la quale mantiene un vettore di direzione costante finché non rileva un ostacolo, per poi

ruotare stocasticamente. Un'ulteriore evoluzione è la **ScoutStrategy2**, che adotta un approccio *Frontier-Based*: l'agente interroga la memoria locale per individuare la coordinata di tipo "ignoto" più vicina, delegando al BFS la rotta per raggiungerla. Pur garantendo una rapida esplorazione individuale, queste logiche sono "cieche" rispetto ai compagni, causando un'elevata duplicazione del lavoro in scenari multi-agente.

- **Navigazione Perimetrale (Wall Follower):** Implementata come primo tentativo di esplorazione strutturata, si basa sulla nota "Regola della Mano Destra", costringendo l'agente a muoversi mantenendo costantemente un ostacolo sul proprio lato destro. Nonostante l'efficacia teorica nei labirinti a corridoio continuo, questa logica è stata valutata e **ufficialmente scartata**. In un ambiente logistico aperto e disseminato di ostacoli centrali (come gli scaffali), la strategia induce gravi inefficienze: gli agenti tendono a bloccarsi in cicli infiniti ridondanti (orbitando attorno a un singolo pilastro) e ignorano gli oggetti o i magazzini visibili se questi non si trovano strettamente adiacenti al muro che stanno seguendo.

- **Dispersione Fisica (Sparce Strategy):** Per ovviare all'accavallamento spaziale, questa strategia introduce una logica di repulsione a corto raggio. L'agente analizza la posizione dei compagni nel suo raggio di prossimità e calcola un "vettore di fuga" sommando le direzioni opposte ai colleghi. La mossa successiva viene scelta massimizzando il prodotto scalare tra le direzioni valide e il vettore di fuga, simulando di fatto un campo magnetico repulsivo che distribuisce uniformemente gli agenti nella stanza.

- **Intelligenza di Sciame (Swarm Explorer):** È la strategia più avanzata e collaborativa del sistema. Invece di limitarsi a schivare i compagni vicini, gli agenti si "spartiscono" in automatico le zone della mappa ancora da visitare, evitando così di esplorare tutti la stessa area. Ad ogni step, l'algoritmo valuta tutte le celle ignote della mappa, assegnando a ciascuna un costo (ovvero un livello di preferenza) basato sulla somma di tre fattori:

  1. **Distanza fisica:** La distanza di Manhattan tra l'agente e la cella ignota.
  2. **Penalità Tabu (+20):** Un malus applicato se la cella rientra nella memoria a breve termine.
  3. **Interferenza dello sciame:** Il fulcro della cooperazione. Se un compagno risulta più vicino alla medesima cella ignota, viene applicata una penalità proibitiva (+100) che forza l'agente corrente a rinunciare a quell'obiettivo. Se il compagno è vicino al target (entro 8 passi), viene applicata una penalità proporzionale.

  L'agente seleziona il target con il costo totale minimo. Questo meccanismo garantisce un'espansione organica, in cui lo sciame si divide spontaneamente ai bivi e copre l'intera planimetria logistica nel minor numero di *tick* possibili.

  Si riporta di seguito il codice `Python` che implementa la funzione di costo per l'allocazione dei target:

```python
best_target = None
best_score = float('inf')

for unk in unknowns:
    my_dist = position.manhattan_distance_to(unk)
    tabu_penalty = 20 if self._is_tabu(unk.x, unk.y) else 0
    teammate_penalty = 0

    # Calcolo dell'interferenza dello sciame
    for t in self.teammates:
        t_dist = abs(t.x - unk.x) + abs(t.y - unk.y)
        if t_dist <= my_dist:
```

```
            teammate_penalty += 100 # Compagno piu' vicino: rinuncia
        elif t_dist < 8:
            teammate_penalty += (8 - t_dist) * 2 # Compagno in avvicinamento

    # Funzione di costo complessiva
    score = my_dist + teammate_penalty + tabu_penalty
    if score < best_score:
        best_score = score
        best_target = unk
```

# 5 Comunicazione e Coordinamento

Il sistema implementa un modello di cooperazione decentralizzata: gli agenti comunicano tra loro scambiandosi le informazioni una volta che i raggi di comunicazione di due o più agenti si intersecano.

L'interazione avviene tramite un protocollo di sincronizzazione della memoria spaziale. In questo caso i due o più agenti si scambiano le informazioni delle proprie mappe locali esplorate, e di eventuali oggetti rilevati ma non recuperati. Questo scambio di dati produce tre vantaggi fondamentali per l'efficienza globale dello sciame:

- Sovrapponendo le matrici delle rispettive mappe locali, gli agenti ampliano istantaneamente la propria visione dell'ambiente. Questo permette loro di scoprire la posizione dei magazzini e la conformazione degli ostacoli senza doverli esplorare fisicamente, riducendo le aree ignote.
- Condividendo lo stato di avanzamento sui target, un agente apprende tempestivamente se un determinato oggetto è già stato prelevato da un collega. Ciò permette di annullare immediatamente le rotte di recupero non più valide, risparmiando inefficienze e consumo energetico legato a viaggi a vuoto.
- L'avvenuta connessione aggiorna la percezione di prossimità dei colleghi. Questa informazione alimenta le euristiche delle logiche esplorative più complesse (come la strategia *Swarm Explorer*), fornendo gli input necessari per calcolare le penalità di interferenza e permettendo agli agenti di spartirsi lo spazio inesplorato senza sovrapporsi.

# 6 Simulazione e Analisi dei Risultati

## 6.1 Setup Sperimentale e Metriche di Valutazione

Per validare l'efficacia del Sistema Multi-Agente e delle strategie esplorative implementate, è stata condotta una vasta campagna di simulazioni. Per garantire la significatività statistica e appianare la varianza dovuta alla generazione stocastica, ogni configurazione è stata sottoposta a **200 iterazioni indipendenti** (100 per ciascuna mappa). I test sono stati condotti mantenendo fissa la dimensione della flotta a 5 agenti operativi. Tale scelta dimensionale verrà motivata nel prosieguo della trattazione, supportata dai risultati di un test comparativo effettuato su uno scenario a 4 agenti.

L'analisi ha monitorato tre metriche principali:

1. **Tasso di Recupero (Success Rate):** Percentuale media di oggetti rilevati, raccolti e depositati.
2. **Tempo di Completamento (Tick):** Iterazioni necessarie per l'esaurimento del task o dell'energia.

3. **Efficienza Energetica:** Consumo medio per agente a fine iterazione.

## 6.2 Confronto prestazionale tra sciami omogenei ed eterogenei

L'esperimento ha valutato sia configurazioni "pure" (flotte composte da agenti cloni con la medesima strategia) sia configurazioni "eterogenee" (squadre miste in cui uno o più agenti adottano una strategia minoritaria per diversificare l'esplorazione). I risultati consolidati sono riassunti nella Tabella 1.

| **Composizione Flotta** | **Mappa** | **Valori Singoli** | | | **Medie Combinate** | | |
|---|---|---|---|---|---|---|---|
| | | **Item (%)** | **Tick** | **Energia** | **Item (%)** | **Tick** | **Energia** |
| *Sciami Omogenei (Flotte Pure)* | | | | | | | |
| ALL Wall Follower | A | 80.80% | 133.88 | 124.96 | *67.75%* | *132.84* | *124.98* |
| | B | 54.70% | 131.79 | 125.00 | | | |
| ALL Scout2 | A | 74.60% | 105.39 | 100.00 | *77.60%* | ***105.16*** | ***99.77*** |
| | B | 80.60% | 104.92 | 99.53 | | | |
| ALL Swarm Explorer | A | 83.10% | 169.36 | 149.35 | *79.70%* | *162.31* | *149.68* |
| | B | 76.30% | 155.26 | 150.00 | | | |
| *Sciami Eterogenei (Es. X Specialisti + Swarm)* | | | | | | | |
| 2 Wall Follower | A | 84.60% | 159.23 | 139.78 | *81.00%* | *157.03* | *139.79* |
| | B | 77.40% | 154.83 | 139.80 | | | |
| 1 Scout2 | A | 84.20% | 166.17 | 139.61 | *83.15%* | *159.70* | *139.01* |
| | B | 82.10% | 153.23 | 138.41 | | | |
| 2 Scout2 | A | 83.70% | 167.50 | 129.37 | *83.55%* | *160.10* | *129.50* |
| | B | 83.40% | 152.70 | 129.63 | | | |
| 1 Wall Follower | A | 85.50% | 162.37 | 144.65 | ***83.90%*** | *158.28* | *144.48* |
| | B | 82.30% | 154.18 | 144.30 | | | |

Tabella 1: Risultati dettagliati per mappa con medie.

## 6.3 Analisi Critica

Dall'analisi dei dati emergono considerazioni fondamentali sul comportamento del sistema multi-agente:

- L'evidente discrepanza dei risultati ottenuti con la strategia *Wall Follower* dimostra come la topologia della mappa ne influenzi fortemente l'efficienza. In ambienti caratterizzati da corridoi brevi e numerose intersezioni, come la mappa B, il tasso di successo crolla al **54.70%**.

- La flotta pura *ALL Scout2* registra i valori più bassi in termini di tick (105.16) e consumo energetico (99.77). Tuttavia, tale velocità è illusoria: agendo in modo egoistico, gli agenti si sovrappongono frequentemente e "svuotano" le aree centrali ignorando gli angoli, fermandosi prematuramente con un tasso di recupero sub-ottimale (**77.60%**).

- La flotta pura *ALL Swarm Explorer* innalza il tasso di successo sfiorando l'80%. Il meccanismo repulsivo vettoriale distribuisce efficacemente gli agenti, ma il continuo tentativo di non intralciarsi a vicenda in corridoi stretti allunga i percorsi e i tempi di manovra (162.31 tick), gravando sul consumo energetico. Anche applicando una variante ottimizzata (*4 Swarm Enhanced*), pur superando l'80% di recupero, si registra il picco massimo di inefficienza temporale (180.97) ed energetica (174.83) a causa dell'eccessivo sovraccarico computazionale e di manovra.

- Il risultato più rilevante risiede nel superamento del paradigma dei "cloni". Inserendo all'interno di una base Swarm un singolo agente con logica differente (es. la configurazione

mista *1 Wall Follower* o *2 Scout2*), il sistema raggiunge la sua **efficienza massima (83.90% di recupero)**. L'agente anomalo agisce come elemento perturbatore: setaccia i perimetri ostici o "taglia" le traiettorie previste, fungendo da "scopritore" caotico per i colleghi senza appesantire il carico cognitivo dell'intero sciame.

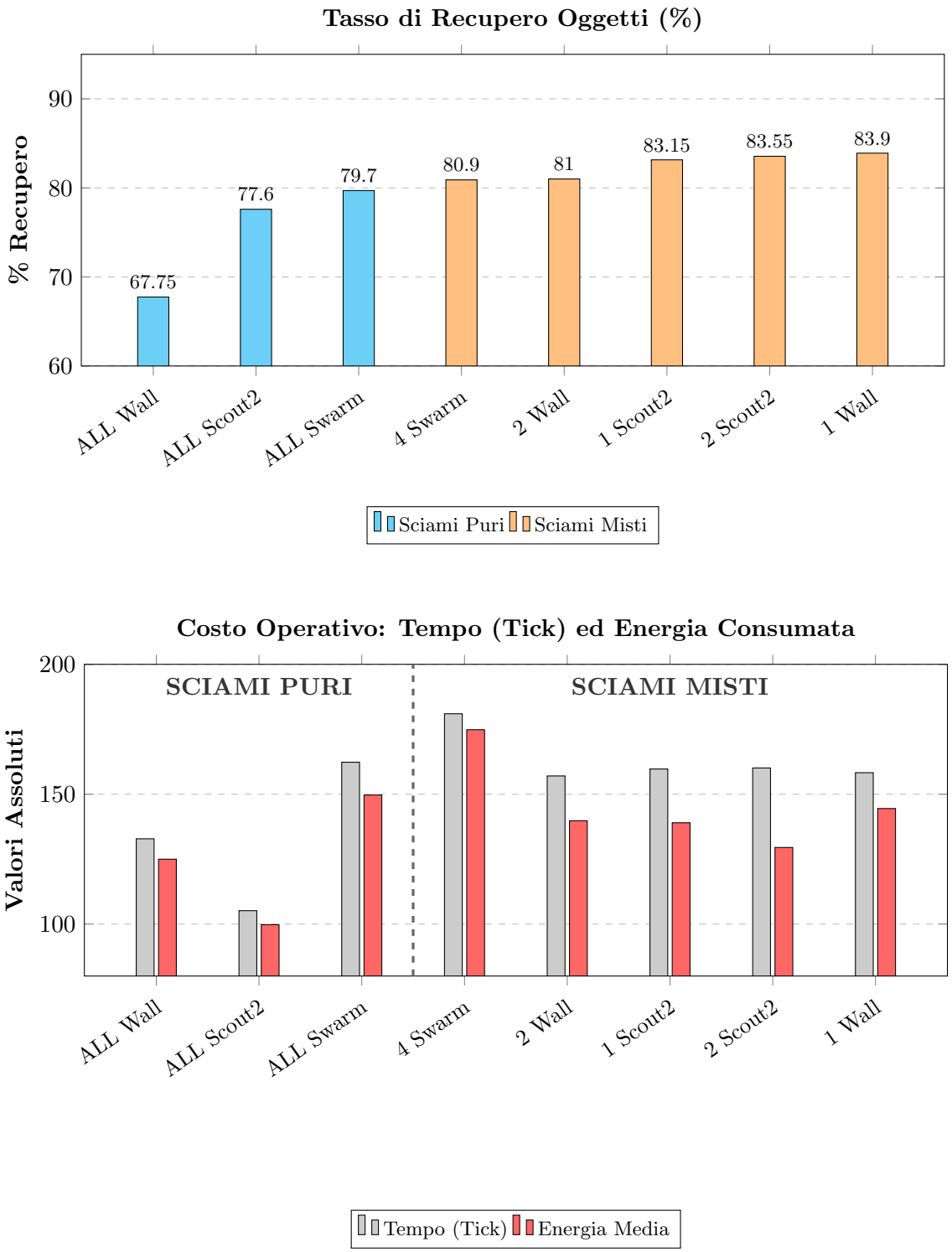


Figura 3: In alto: Confronto del Tasso di Recupero tra le 8 configurazioni testate. In basso: Costi operativi in termini di iterazioni ed energia. Si nota come le soluzioni miste garantiscano il miglior compromesso tra successo e consumo logistico.

### 6.4 Ottimizzazione del numero di agenti

La scelta di impostare la flotta a 5 agenti nasce principalmente per mantenere i tempi di completamento intorno o sotto i 150 tick. Per averne la certezza, è stato fatto un test riducendo la squadra a 4 robot (*4 Swarm*) e distribuendo l'energia dell'agente mancante tra quelli rimasti in gioco. I risultati, però, sono chiaramente peggiorati: il tempo medio è salito a quasi 181 tick, mancando in pieno l'obiettivo. Oltre a impiegare più tempo, i 4 agenti hanno consumato molta più energia a testa (174.83) e la percentuale di oggetti recuperati è scesa all'80.90%. Questo conferma che 5 robot è il numero ideale per dividersi il lavoro in modo veloce ed efficiente. D'altra parte, è essenziale non eccedere con le unità per evitare il sovraffollamento degli spazi: un aumento sconsiderato degli agenti moltiplicherebbe gli stalli e le manovre evasive, rallentando pesantemente l'intera esplorazione concorrente.

## 7 Conclusioni e Sviluppi Futuri

Il presente progetto ha permesso di modellare, implementare e analizzare un Sistema Multi-Agente complesso per l'ottimizzazione logistica. Sviluppando e testando diverse euristiche di navigazione, si è dimostrato come l'impiego di una logica decentralizzata consenta a una flotta di robot di mappare un ambiente ignoto e portare a termine task di recupero senza alcun controllo centrale.

Il risultato più significativo dell'analisi prestazionale risiede nella validazione del concetto di **Sciame Eterogeneo**. Gli esperimenti hanno provato scientificamente che l'inserimento di una percentuale di diversità algoritmica (come un agente con logica *Wall Follower* all'interno di un team *Swarm Explorer*) permette di rompere i cicli di stallo tipici dei sistemi cooperativi omogenei. Questo approccio ibrido ha garantito il miglior bilanciamento multicriterio, massimizzando il tasso di recupero degli oggetti senza gravare sui costi operativi di tempo ed energia.

### 7.1 Sviluppi Futuri

Nonostante la solidità dell'architettura attuale, la natura modulare del codice si presta a diverse evoluzioni concettuali per incrementare ulteriormente l'efficienza:

- **Adattatività Dinamica delle Strategie:** Attualmente, un agente mantiene la medesima logica per tutta la durata della simulazione. Un'evoluzione naturale consisterebbe nell'implementare transizioni di strategia basate sullo stato interno o sul contesto. Ad esempio, gli agenti potrebbero iniziare con un approccio repulsivo (*Sparce Strategy*) per massimizzare la copertura iniziale dell'area. Successivamente, al calare dell'energia sotto una soglia critica (es. 50%), potrebbero commutare su logiche attrattive, convergendo intenzionalmente verso i compagni per fondere le mappe prima di esaurire le batterie.

- **Specializzazione di Ruolo:** Si potrebbe superare il concetto di flotta generalista introducendo classi di agenti con obiettivi asimmetrici. Inserire una quota di agenti "Scout" puri, programmati per ignorare la raccolta fisica degli oggetti (disabilitando lo stato *REACH_ITEM*) e focalizzati unicamente sull'espansione della mappa e sulla scoperta rapida dei magazzini. Questi agenti agirebbero come radar per lo sciame, comunicando le coordinate scoperte ai "Worker", i quali ottimizzerebbero le proprie traiettorie di raccolta e consegna senza disperdere energia in percorsi di ricerca cieca.